

```
In [ ]: # 5.2 Градиентный бустинг (регрессия).
```

```
In [18]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в заданиях 1.2, 2.2, 3.2, 4.2.
```

```
In [19]: df_air = pd.read_csv(
    'C:/Users/Elizaveta/Downloads/air_quality/AirQuality.csv',
    delimiter=';',
    decimal=',',
    na_values=[' ', 'NA', 'nan', -200]
)
print(f"размер после загрузки: {df_air.shape}")
print("столбцы после загрузки:", list(df_air.columns))
df_air = df_air.loc[:, ~df_air.columns.str.contains('Unnamed')]
print(f"\nразмер таблицы после удаления пустых столбцов: {df_air.shape}")
print("столбцы:", list(df_air.columns))
print(df_air.head())
print(df_air.info())
print(df_air.isnull().sum())
```

размер после загрузки: (9471, 17)

столбцы после загрузки: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH', 'Unnamed: 15', 'Unnamed: 16']

размер таблицы после удаления пустых столбцов: (9471, 15)

столбцы: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	\
0	10/03/2004	18.00.00	2.6	1360.0	150.0	11.9	
1	10/03/2004	19.00.00	2.0	1292.0	112.0	9.4	
2	10/03/2004	20.00.00	2.2	1402.0	88.0	9.0	
3	10/03/2004	21.00.00	2.2	1376.0	80.0	9.2	
4	10/03/2004	22.00.00	1.6	1272.0	51.0	6.5	

	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	\
0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	
1	955.0	103.0	1174.0	92.0	1559.0	972.0	
2	939.0	131.0	1140.0	114.0	1555.0	1074.0	
3	948.0	172.0	1092.0	122.0	1584.0	1203.0	
4	836.0	131.0	1205.0	116.0	1490.0	1110.0	

	T	RH	AH
0	13.6	48.9	0.7578
1	13.3	47.7	0.7255
2	11.9	54.0	0.7502
3	11.0	60.0	0.7867
4	11.2	59.6	0.7888

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9471 entries, 0 to 9470

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	9357 non-null	object
1	Time	9357 non-null	object
2	CO(GT)	7674 non-null	float64
3	PT08.S1(CO)	8991 non-null	float64
4	NMHC(GT)	914 non-null	float64
5	C6H6(GT)	8991 non-null	float64
6	PT08.S2(NMHC)	8991 non-null	float64
7	NOx(GT)	7718 non-null	float64
8	PT08.S3(NOx)	8991 non-null	float64
9	NO2(GT)	7715 non-null	float64
10	PT08.S4(NO2)	8991 non-null	float64
11	PT08.S5(O3)	8991 non-null	float64
12	T	8991 non-null	float64
13	RH	8991 non-null	float64
14	AH	8991 non-null	float64

dtypes: float64(13), object(2)

memory usage: 1.1+ MB

None

Date 114

Time 114

CO(GT) 1797

PT08.S1(CO) 480

NMHC(GT) 8557

C6H6(GT) 480

PT08.S2(NMHC) 480

NOx(GT) 1753

```

PT08.S3(NOx)      480
NO2(GT)           1756
PT08.S4(NO2)      480
PT08.S5(O3)       480
T                 480
RH                480
AH                480
dtype: int64

```

```

In [20]: X = df_air[feature_columns]
Y = df_air[target_column]
mask = X.notnull().all(axis=1)&Y.notnull()
X_clean = X[mask]
Y_clean = Y[mask]
print(f"Размер X: {X_clean.shape}")
print(f"Размер Y: {Y_clean.shape}")
print(f"\nЗначения целевой переменной {target_column}:")
print(f"Min: {Y_clean.min():.3f}")
print(f"Max: {Y_clean.max():.3f}")
print(f"Среднее: {Y_clean.mean():.3f}")
print(f"Медиана: {Y_clean.median():.3f}")
print(f"Стандартное отклонение: {Y_clean.std():.3f}")

```

```

Размер X: (6941, 11)
Размер Y: (6941,)

```

```

Значения целевой переменной C6H6(GT):
Min: 0.200
Max: 63.700
Среднее: 10.554
Медиана: 8.800
Стандартное отклонение: 7.465

```

```

In [21]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean,
                                                    test_size=0.3,
                                                    random_state=42)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")

```

```

Обучающая выборка: (4858, 11)
Тестовая выборка: (2083, 11)

```

```

In [22]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean, test_size=0.3,
                                                    random_state=42)
print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")

```

```

Обучающая выборка: (4858, 11)
Тестовая выборка: (2083, 11)

```

```

In [ ]: ## 2. Создание бейзлайна.

Ниже представлено построение базовой модели случайного леса RandomForestRegressor.
Обучающая выборка (train) используется для обучения модели.
Тестовая выборка (test) используется для оценки качества обученной модели.
Используются метрики MAE, RMSE, R2.
MAE=0.0585 показывает среднюю величину ошибки в тех же единицах, что и целевая переменная.
RMSE=0.2884 также показывает среднюю ошибку переменной, но за счёт возведения в квадрат
демонстрирует наличие выбросов.
R2=0.9985 показывает долю дисперсии целевой переменной, полученное значение под
Базовая модель демонстрирует высокие показатели, возможно переобучение. Она служ

```

```
In [8]: from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
baseline_gb_reg = GradientBoostingRegressor(random_state=42)
baseline_gb_reg.fit(X_train, Y_train)
Y_baseline_gb_reg = baseline_gb_reg.predict(X_test)
MAE = mean_absolute_error(Y_test, Y_baseline_gb_reg)
RMSE = np.sqrt(mean_squared_error(Y_test, Y_baseline_gb_reg))
R2 = r2_score(Y_test, Y_baseline_gb_reg)
print(f" MAE={MAE:.4f}, RMSE={RMSE:.4f}, R2={R2:.4f}")
```

MAE=0.0585, RMSE=0.2884, R2=0.9985

```
In [ ]: ## 3. Улучшение бейзлайна.
Гипотеза 1: Подбор гиперпараметров.
Для улучшения модели подберем гиперпараметры с помощью RandomizedSearchCV: колич
ограничения на разбиение и размер листьев, доля выборки, используемая для постро
функция потерь. Используется 50 комбинаций с 5-кратной кросс-валидацией.
Результаты:
Лучшие параметры (регрессия): {'subsample': 1.0, 'n_estimators': 100, 'min_sampl
Лучший отрицательный MSE на CV: -0.01220931925766293
Улучшенный регрессор: MAE = 0.0470, RMSE = 0.2971, R2 = 0.9985
полученные результаты говорят об уменьшении количества ошибок.
```

```
In [9]: from sklearn.model_selection import RandomizedSearchCV
param_dist_reg = {
    'n_estimators': [100, 200, 300, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'subsample': [0.8, 1.0],
    'max_features': ['sqrt', 'log2', 0.5, 1.0],
    'loss': ['squared_error', 'absolute_error', 'huber']
}
gb_reg = GradientBoostingRegressor(random_state=42)
random_search_reg = RandomizedSearchCV(
    gb_reg,
    param_distributions=param_dist_reg,
    n_iter=50,
    cv=5,
    scoring='neg_mean_squared_error',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search_reg.fit(X_train, Y_train)
print("Лучшие параметры:", random_search_reg.best_params_)
print("Лучший отрицательный MSE на CV:", random_search_reg.best_score_)
best_gb_reg = random_search_reg.best_estimator_
y_pred_reg = best_gb_reg.predict(X_test)
mae = mean_absolute_error(Y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(Y_test, y_pred_reg))
r2 = r2_score(Y_test, y_pred_reg)
print(f"Улучшенный регрессор: MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Лучшие параметры (перекрестная): {'subsample': 1.0, 'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 1.0, 'max_depth': 7, 'loss': 'squared_error', 'learning_rate': 0.05}

Лучший отрицательный MSE на CV: -0.01220931925766293

Улучшенный регрессор: MAE = 0.0470, RMSE = 0.2971, R2 = 0.9985

In []: *## 4. Имплементация алгоритма машинного обучения.*

Был создан собственный алгоритм градиентного бустинга, построенный на основе алг

```
In [10]: import numpy as np
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value
class MyDecisionTreeRegressor:
    def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1, random_state=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.random_state = random_state
        self.root = None
        self.n_features_ = None
        self.rng = np.random.RandomState(random_state)
    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        self.n_features_ = X.shape[1]
        self.root = self._build_tree(X, y, depth=0)
        return self
    def _build_tree(self, X, y, depth):
        n_samples = X.shape[0]
        if (depth == self.max_depth) or (n_samples < self.min_samples_split) or (n_samples < self.min_samples_leaf):
            leaf_value = np.mean(y)
            return Node(value=leaf_value)
        best_feat, best_thresh, best_gain = self._best_split(X, y)
        if best_feat is None:
            leaf_value = np.mean(y)
            return Node(value=leaf_value)
        left_mask = X[:, best_feat] <= best_thresh
        right_mask = ~left_mask
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
        return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child)
    def _best_split(self, X, y):
        best_gain = -float('inf')
        best_feature = None
        best_threshold = None
        current_var = np.var(y)
        n_features = X.shape[1]
        for feature in range(n_features):
            thresholds = np.unique(X[:, feature])
            for thresh in thresholds:
                left_mask = X[:, feature] <= thresh
                right_mask = ~left_mask
                if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mask) < self.min_samples_leaf:
                    continue
                left_mean = np.mean(y[left_mask])
                right_mean = np.mean(y[right_mask])
                gain = current_var - (np.sum(left_mask) * left_mean**2 + np.sum(right_mask) * right_mean**2) / n_samples
                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature
                    best_threshold = thresh
```

```

        left_var = np.var(y[left_mask])
        right_var = np.var(y[right_mask])
        n_left = np.sum(left_mask)
        n_right = np.sum(right_mask)
        n_total = n_left + n_right
        gain = current_var - (n_left / n_total) * left_var - (n_right /
        if gain > best_gain:
            best_gain = gain
            best_feature = feature
            best_threshold = thresh
    if best_gain <= 0:
        return None, None, None
    return best_feature, best_threshold, best_gain
def predict(self, X):
    X = np.array(X)
    predictions = []
    for x in X:
        node = self.root
        while node.value is None:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        predictions.append(node.value)
    return np.array(predictions)

```

```

In [13]: class MyGradientBoostingRegressor:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3,
                  min_samples_split=2, min_samples_leaf=1, subsample=1.0,
                  random_state=None):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.subsample = subsample
        self.random_state = random_state
        self.trees = []
        self.initial_pred_ = None
        self.rng = np.random.RandomState(random_state)
    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        self.initial_pred_ = np.mean(y)
        F = np.full(len(y), self.initial_pred_)
        for i in range(self.n_estimators):
            residuals = y - F
            if self.subsample < 1.0:
                n_samples = int(self.subsample * len(y))
                indices = self.rng.choice(len(y), size=n_samples, replace=False)
                X_sub = X[indices]
                residuals_sub = residuals[indices]
            else:
                X_sub = X
                residuals_sub = residuals
            tree = MyDecisionTreeRegressor(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split,
                min_samples_leaf=self.min_samples_leaf,
                random_state=self.rng.randint(0, 10**6)

```

```

    )
    tree.fit(X_sub, residuals_sub)
    self.trees.append(tree)
    F += self.learning_rate * tree.predict(X)
    return self
def predict(self, X):
    X = np.array(X)
    F = np.full(len(X), self.initial_pred_)
    for tree in self.trees:
        F += self.learning_rate * tree.predict(X)
    return F

```

In []: Тестирование собственного алгоритма. Результаты:

```

MAE = 0.0321
RMSE = 0.2774
R2 = 0.9987

```

Полученные метрики показали качество распознавания выше, чем у библиотечной моде

```

In [23]: my_gb_reg = MyGradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    min_samples_split=2,
    min_samples_leaf=1,
    subsample=0.8,
    random_state=42
)
my_gb_reg.fit(X_train, Y_train)
y_pred_reg = my_gb_reg.predict(X_test)
mae = mean_absolute_error(Y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(Y_test, y_pred_reg))
r2 = r2_score(Y_test, y_pred_reg)
print("MyGradientBoostingRegressor")
print(f"MAE = {mae:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"R2 = {r2:.4f}")

```

```

MyGradientBoostingRegressor
MAE = 0.0321
RMSE = 0.2774
R2 = 0.9987

```

In []: ##Выводы:

В ходе исследования градиентного бустинга для регрессии была создана базовая мод (MAE=0.0585, RMSE=0.2884, R2=0.9985), так и после подбора гиперпараметров (MAE собственный алгоритм градиентного бустинга для регрессии, основанный на алгоритм данного алгоритма значения метрик (MAE = 0.0321, RMSE = 0.2774, R2 = 0.9987) пр корректной реализации.

##Подведение итогов и анализ результатов: В ходе выполнения работы были исследованы 5 алгоритмов машинного обучения: KNN, линейные модели, решающие деревья, случайный лес, градиентный бустинг. Для каждого алгоритма выполнялись этапы построения базовой модели, улучшения, разработки и тестирования собственной реализации. Для классификации наилучший результат (F1=0.9148) показал градиентный бустинг, незначительно опередив случайный лес (F1=0.9148). Решающее дерево также показало высокий результат (F1=0.8904). Далее идёт логистическая регрессия с полиномиальными признаками (F1=0.8808). На последнем месте оказался алгоритм KNN (F1=0.8377). Для регрессии все алгоритмы показали высокое качество (R2>0.96) благодаря сильным корреляциям между признаками и целевой переменной, даже после удаления самого информативного признака. Наиболее высокий показатель наблюдается у случайного леса (0.9984), а самый меньший - у KNN (0.9622). Таким образом, наилучшие результаты в классификации продемонстрировали ансамблевые методы (градиентный бустинг и случайный лес), а в регрессии - случайный лес и решающее дерево. Алгоритм KNN оказался наименее эффективным из всех. Предобработка данных (масштабирование, удаление сильно коррелированных признаков, подбор гиперпараметров) является критически

важным этапом, который помогает алгоритмам продемонстрировать свой потенциал. Разработанные собственные алгоритмы успешно прошли сравнение с библиотечными аналогами, показав в большинстве случаев идентичные результаты метрик.

In []: